
CS 421 --- State Monad Activity

Manager	Keeps team on track	
Recorder	Records decisions / QC	
Reporter	Reports to class	
Reflector	Assesses team performance	

Please write your name/netid legibly in dark ink. Hand in one copy per team. Do not staple or mangle the corners.

The State Monad

```
0 data State s a = State { runState :: s -> (a,s) }
1
2 instance Monad (State s) where
3   return = pure -- or ... return a = State (\s -> (a,s))
4   x >>= f = State (\s -> let (y,s2) = runState x s
5                           (z,s3) = runState (f y) s2
6                           in (z,s3))
7
8 get :: State s s
9 get = State (\s -> (s,s))
10
11 put :: a -> State a ()
12 put x = State (\s -> ((),x))
13
14 newState a = State (\s -> (a,s))
```

Problem 1) Notice how when we call pure, we return a State function that does not use its state at all. Why is that the right thing to do?

Problem 2) What does the syntax `runState x s` mean?

Problem 3) What is the type of the expression `(f y)`? Why does it have to be that type?

Problem 4) We call `runState` a second time on `(f y)`. We use `s2` in this case. What would happen if we used `s` instead?

Problem 5) Explain what `get` and `put` are doing. Make sure everyone on the team understands them.

Using the State Monad

Here are the Functor and Applicative definitions for State, for reference.

```
0 instance Functor (State s) where
1   fmap f x = State (\s -> let (y,s2) = runState x s
2                           in (f y, s2))
3
4 instance Applicative (State s) where
5   pure a = State (\s -> (a,s))
6   ff <*> xx = State (\s -> let (f,s2) = runState ff s
7                           (x,s3) = runState xx s2
8                           in (f x, s3))
```

Problem 6) Write a function `cplus :: Num a => State s a -> State s a -> State s a` that takes two state integers and adds them, also incrementing the state.

```
0 Prelude> Main.runState (cplus (newState 10) (newState 20)) 0
1 (30,1)
```

Problem 7) `get` and `put` are boring. Write `push :: a -> State [a] ()` and `pop :: State [a] a`. You can use `get` and `put` in your definition if you want. Here is a sample function that uses it.

```
0 addStack x = do
1   a <- x
2   b <- pop
3   push (a + b)
4   return b
5
6 Prelude> Main.runState (addStack (newState 10)) [5,6]
7 (5,[15,6])
```

State Monad Activity--- Reflector's Report

Manager	Keeps team on track	
Recorder	Records decisions	
Reporter	Reports to Class	
Reflector	Assesses team performance	

1. What was a strength of your team's performance for this activity?
2. What could you do next time to increase your team's performance?
3. What insights did you have about the activity or your team's interaction today?

State Monad Activity --- Team's Assessment (SII)

Manager or Reflector: Consider the objectives of this activity and your team's experience with it, and then answer the following questions after consulting with your team.

1. What was a **strength** of this activity? List one aspect that helped it achieve its purpose.
2. What is one thing we could do to **improve** this activity to make it more effective?
3. What **insights** did you have about the activity, either the content or at the meta level?